

Trabalho Pratico 2 - FPAA

Aluno: Lucas Picinin

Matricula: 689393

Repositorio: <https://github.com/Lucas-Lutti/FPAA-Atividade-2>

1. Modelagem do problema

O tabuleiro foi representado por uma matriz quadrada de tamanho par. Cada posicao pode estar vazia ou conter um dos dois simbolos do jogo:

Simbolo no jogo	Entrada usada no programa	Valor interno
 Lua	L	0
 Sol	S	1
Celula vazia	.	-1

```
-1 = vazio  
0 = Lua  
1 = Sol
```

As restricoes entre celulas foram guardadas em duas matrizes separadas. A matriz horizontal armazena relacoes entre celulas vizinhas da mesma linha. A matriz vertical armazena relacoes entre uma celula e a celula logo abaixo dela.

```
0 = sem restricao  
1 = igualdade  
-1 = oposicao
```

Em um tabuleiro $n \times n$, a matriz de restricoes horizontais tem n linhas e $n - 1$ colunas. A matriz de restricoes verticais tem $n - 1$ linhas e n colunas. Essa separacao facilita a validacao, porque cada tipo de relacao e consultado diretamente na posicao onde ela aparece.

No codigo, essa modelagem aparece principalmente na estrutura `Tabuleiro`. Ela guarda tamanho, celulas, `restricoesHorizontais` e `restricoesVerticais`. A validacao das regras fica na classe `Validador`, enquanto as buscas ficam na classe `Resolvedor`.

O programa le os dados a partir de um arquivo de texto. A primeira parte contem as celulas iniciais, usando S para Sol, L para Lua e . para vazio. Depois aparecem as restricoes horizontais, marcadas por H, e as restricoes verticais, marcadas por V.

2. Estrategia de resolucao por forza bruta

Na força bruta, o algoritmo identifica todas as casas vazias do tabuleiro e gera todas as possibilidades de preenchimento para essas casas. Para cada casa vazia existem duas escolhas possíveis: Sol ou Lua. Assim, se o tabuleiro tem k casas vazias, o total de combinações possíveis é 2^k .

O ponto principal da força bruta é que ela não tenta perceber erros durante a construção da resposta. O algoritmo preenche todas as lacunas e somente quando o tabuleiro está completo chama a função de validação final. Essa validação confere:

- se todas as células foram preenchidas;
- se não existem três símbolos iguais consecutivos na horizontal ou na vertical;
- se cada linha tem a mesma quantidade de Sois e Luas;
- se cada coluna tem a mesma quantidade de Sois e Luas;
- se as restrições de igualdade e oposição foram respeitadas.

Essa estratégia é simples e ajuda a mostrar o tamanho do espaço de busca. Por outro lado, ela rapidamente fica cara quando há muitas casas vazias.

3. Estratégia de resolução por backtracking

O backtracking também testa valores nas casas vazias, mas não espera o tabuleiro inteiro ficar pronto para verificar se houve erro. Depois de cada tentativa, o programa aplica uma validação parcial. Se alguma regra já foi quebrada, aquela tentativa é abandonada imediatamente.

A função recursiva segue esta ideia:

1. Escolher uma célula vazia.
2. Testar Lua.
3. Validar parcialmente o tabuleiro.
4. Continuar a recursão se a tentativa ainda for possível.
5. Caso contrário, desfazer a escolha.
6. Repetir o processo com Sol.

A condição de parada acontece quando não existem mais células vazias. Nesse momento, o algoritmo chama a validação completa. Se o tabuleiro final respeita todas as regras, a solução foi encontrada.

As podas usadas no backtracking são:

- rejeitar linhas ou colunas que já passaram da metade permitida de Sois ou Luas;
- rejeitar três símbolos iguais consecutivos;
- rejeitar uma linha ou coluna completa que não esteja equilibrada;
- rejeitar restrições = violadas;
- rejeitar restrições x violadas.

O programa também escolhe, a cada passo, uma casa vazia com menor quantidade de valores possíveis naquele momento. Isso não muda as regras do problema, apenas reduz tentativas desnecessárias.

4. Exemplos de execução

Compilacao:

```
make
```

Execucao com os dois algoritmos:

```
build/tango exemplos/exemplo_facil_6x6.txt ambos
```

Recorte da saida obtida no exemplo 6x6:

```
Algoritmo: forca-bruta
Chamadas recursivas: 55
Tabuleiros completos testados: 27
Tempo: 0.000002s
Solucao encontrada:
S = S   L   L x S x L
x  x   x   x   x   x
L = L   S   S x L x S
x  =   =   x   =   =
S x L   S   L   L x S
x  x   x   x   x   x
L x S x L x S = S x L
x  x   =   =   x   x
S x L = L x S x L x S
x  x   x   x   x   x
L x S = S x L x S x L

Algoritmo: backtracking
Chamadas recursivas: 6
Tabuleiros completos testados: 1
Tempo: 0.000004s
Solucao encontrada:
S = S   L   L x S x L
x  x   x   x   x   x
L = L   S   S x L x S
x  =   =   x   =   =
S x L   S   L   L x S
x  x   x   x   x   x
L x S x L x S = S x L
x  x   =   =   x   x
S x L = L x S x L x S
x  x   x   x   x   x
L x S = S x L x S x L
```

Execucao apenas com backtracking em uma entrada transcrita a partir de um print:

```
build/tango exemplos/exemplo_print_6x6.txt backtracking
```

Recorte da saida:

```

Tabuleiro inicial:
.  S  S  .  .  S
.  .  .  .  .  .
.  L  .  S  .  L
.  .  S  .  S  .
S  .  .  .  .  L
.  L  .  .  .  .

Algoritmo: backtracking
Chamadas recursivas: 26
Tabuleiros completos testados: 1
Solucao encontrada:
L  S  S  L  L  S
L  S  L  S  S  L
S  L  S  S  L  L
L  L  S  L  S  S
S  S  L  L  S  L
S  L  L  S  L  S

```

Esses recortes mostram que a mesma estrutura de entrada pode representar tanto um tabuleiro quase completo quanto uma instancia copiada visualmente de um jogo.

5. Analise de complexidade

Se existem k celulas vazias, a forza bruta pode precisar avaliar ate 2^k combinacoes. Cada combinacao completa ainda precisa passar pela validacao das regras do jogo. Para um tabuleiro $n \times n$, essa validacao percorre linhas, colunas e restricoes, ficando em torno de $O(n^2)$. Assim, a ordem geral da forza bruta pode ser vista como:

$$O(2^k * n^2)$$

O backtracking tambem possui pior caso exponencial, porque em uma situacao muito desfavoravel pode ser necessario testar muitas combinacoes. A diferenca pratica e que as podas reduzem bastante a arvore de busca. Quando uma linha ja tem Sois demais, quando aparecem tres simbolos iguais seguidos ou quando uma restricao e quebrada, o algoritmo descarta aquele caminho sem completar o restante do tabuleiro.

Portanto, a complexidade teorica continua exponencial no pior caso, mas o numero real de estados visitados tende a ser bem menor do que na forza bruta pura.

6. Conclusao

O trabalho mostra a diferenca entre testar todas as possibilidades e usar as regras do problema durante a busca. A forza bruta e direta, mas desperdica muitas tentativas em tabuleiros que ja poderiam ser descartados antes. O backtracking usa as mesmas regras do Tango como criterios de poda, tornando a resolucao mais eficiente e mais adequada para tabuleiros com muitas lacunas.